

Sistema Distribuido para Comparación de Precios en Supermercados de Chile

Eduardo Leal¹, Benjamín Martínez¹, Luis Olivares¹, and Ninoska Toledo¹

Universidad Austral de Chile, Valdivia, Chile

eduardo.leal@alumnos.uach.cl, benjamin.martinez@alumnos.uach.cl, luis.olivares@alumnos.uach.cl,
ninoska.toledo@alumnos.uach.cl

Resumen Este documento presenta el diseño de un sistema distribuido orientado a la recopilación automática, consolidación y visualización de precios de productos en las principales cadenas de supermercados de Chile (Jumbo, Líder y Unimarc). La arquitectura propuesta sigue el patrón publicador/suscriptor: scrapers independientes recolectan precios periódicamente y los publican en un broker de mensajes (RabbitMQ [5]); un servicio agregador consume la cola, persiste los datos en PostgreSQL 16 [7] y expone una API REST construida con FastAPI [6]; un frontend en React permite a los usuarios buscar y comparar precios entre tiendas en tiempo real, con acceso a historial de variaciones. Todo el sistema opera bajo tecnología de contenedores Docker [1] sobre herramientas de software completamente libre

Keywords: sistema distribuido · scraping web · comparación de precios · RabbitMQ · Docker · FastAPI · PostgreSQL

1. Introducción

El acceso a información de precios en supermercados representa un desafío cotidiano para los consumidores chilenos. Las principales cadenas —Jumbo, Líder y Unimarc— publican sus catálogos en sitios web independientes y sin integración entre ellos, obligando al usuario a realizar comparaciones manuales que consumen tiempo y que frecuentemente se basan en información desactualizada.

Los sistemas distribuidos ofrecen una solución natural a este problema: permiten recolectar datos desde múltiples fuentes de forma paralela, consolidarlos en un repositorio centralizado y exponerlos mediante una interfaz unificada. El presente proyecto aplica estos principios para construir una plataforma de comparación de precios en tiempo real, utilizando el patrón publicador/suscriptor como modelo de comunicación entre componentes, tecnología de contenedores Docker para el despliegue y herramientas de software completamente libre para minimizar costos operacionales.

El documento está organizado de la siguiente forma: la Sección 2 describe el problema detectado, la Sección 3 presenta la solución propuesta, las Secciones 4 a 7 detallan los distintos modelos del diseño, y las Secciones 8 a 10 abordan las decisiones tecnológicas y los supuestos del proyecto.

2. Trabajo Relacionado

Existen plataformas internacionales como Google Shopping, PriceSpy e Idealo que comparan precios mediante APIs, pero se enfocan en mercados europeos y norteamericanos. En Chile, herramientas como Knasta presentan limitaciones en cobertura, actualización y acceso a datos, debido a la falta de APIs públicas, lo que obliga al uso de scraping.

A nivel teórico, trabajos como el de Kitchin [3] sobre infraestructuras de datos y Kleppmann [4] sobre sistemas de procesamiento de datos respaldan el uso de arquitecturas con mensajería. El patrón Pub/Sub está ampliamente documentado en [2].

La propuesta de este trabajo se diferencia al combinar: (1) scraping distribuido con nodos independientes por tienda, (2) comunicación asíncrona vía cola de mensajes, (3) persistencia con historial completo de precios, y (4) despliegue íntegramente en contenedores Docker sobre infraestructura propia.

3. Metodología y Solución Propuesta

3.1. Arquitectura General

La solución sigue el patrón Publicador/Suscriptor sobre una cola de mensajes [2]. Este patrón desacopla los productores de datos (scrapers) del consumidor (agregador), permitiendo escalar cada componente de forma independiente y tolerar caídas transitorias sin pérdida de mensajes.

La Figura 1 muestra el diagrama de arquitectura del sistema. Los scrapers publican mensajes JSON al broker RabbitMQ; el Servicio Agregador consume la cola, valida con Pydantic y persiste en PostgreSQL; la API REST expone los datos al frontend React, el cual es accedido por el usuario a través de Traefik

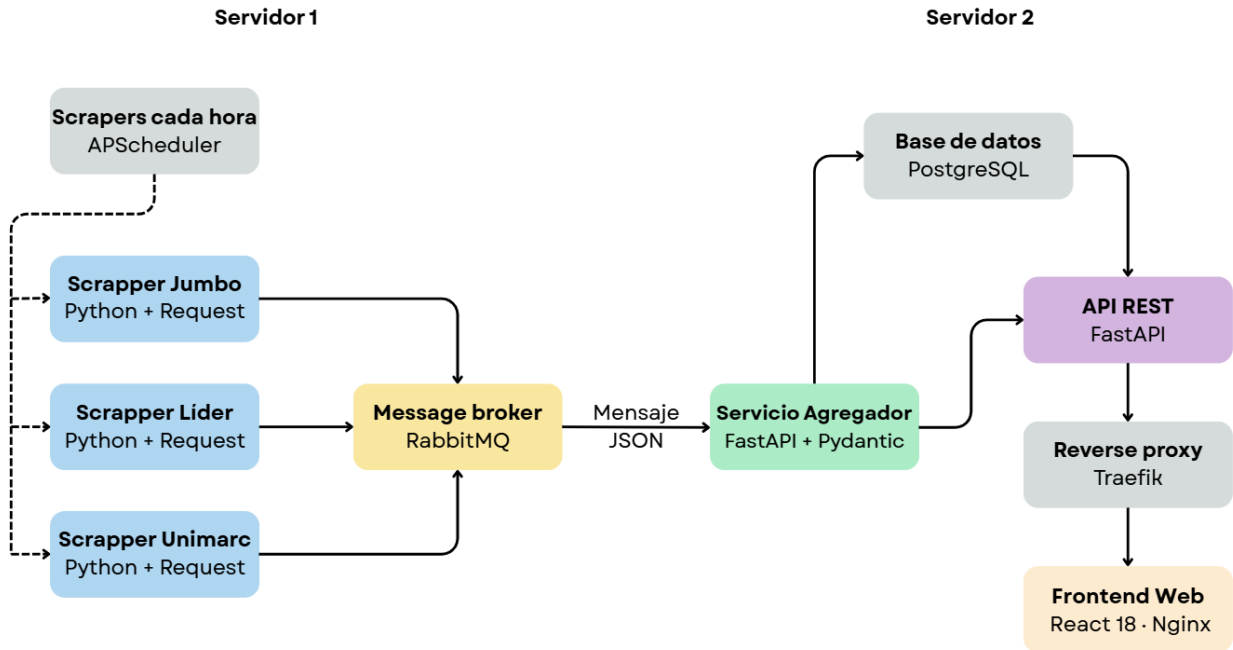


Figura 1. Diagrama de arquitectura

3.2. Componentes del Sistema

El Cuadro 1 detalla cada componente de software, su versión y su rol dentro de la arquitectura.

3.3. Modelo Físico

La infraestructura contempla dos servidores físicos con las siguientes características:

- **Servidor 1:** 8 cores, 16 GB RAM, 500 GB SSD, Ubuntu 22.04 LTS. Aloja los tres scrapers (Jumbo, Líder, Unimarc), APScheduler y el broker RabbitMQ, todos en contenedores Docker independientes. Este servidor actúa como capa de recolección de datos.
- **Servidor 2:** 8 cores, 16 GB RAM, 500 GB SSD, Ubuntu 22.04 LTS. Aloja el Servicio Agregador, la API REST, PostgreSQL 16, el frontend React servido vía Nginx y Traefik como reverse proxy. Todo en contenedores Docker.

La comunicación entre servidores usa la red interna de la organización. El Servidor 2 expone únicamente los puertos 80 (HTTP) y 443 (HTTPS) hacia Internet

Componente	Versión	Descripción / Rol
Python	3.11	Lenguaje base de todos los servicios del sistema
Docker	26.0	Containerización de cada servicio independiente
Docker Compose	2.26	Orquestación local multi-container en cada servidor
APScheduler	3.10	Scheduler: dispara scrapers cada hora automáticamente
RabbitMQ	3.12	Broker de mensajes: cola de precios entre scrapers y agregador
PostgreSQL	16	Base de datos relacional centralizada en Servidor 2
Nginx	1.25	Servidor estático para el frontend React
Traefik	2.11	Reverse proxy y API Gateway (puertos 80/443)
React	18	Interfaz web de búsqueda y comparación de precios

Cuadro 1. Componentes del sistema

3.4. Modelo Fundamental

El sistema sigue el modelo fundamental **cliente-servidor** combinado con el patrón **Publicador/Suscriptor** (Pub/Sub) [2]. Los roles se distribuyen de la siguiente forma:

- **Productores (scrapers):** actúan como clientes que generan datos. Cada scraper es independiente y publica mensajes JSON en el broker sin conocer quién los consume.
- **Broker (RabbitMQ):** componente central que desacopla productores y consumidor. Garantiza persistencia de mensajes ante caídas transitorias del agregador.
- **Consumidor (Servicio Agregador):** suscriptor que procesa los mensajes, valida su esquema y persiste los datos en PostgreSQL.
- **Clientes REST (Frontend):** consultan la API REST del Servidor 2; no interactúan directamente con la cola ni con la base de datos.

Este modelo garantiza bajo acoplamiento entre componentes: un scraper puede fallar sin afectar a los demás ni al agregador, y nuevos scrapers pueden incorporarse publicando en la misma cola sin modificar la arquitectura existente.

3.5. Modelo Relacional y Diccionario de Datos

La base de datos PostgreSQL contiene tres tablas principales: supermercados, productos y precios. La tabla precios registra cada lectura con su marca temporal, habilitando consultas históricas. La Figura 2 presenta el diccionario de datos completo.

3.6. Bibliotecas Python

- **requests:** Peticiones HTTP a los sitios web de supermercados
- **beautifulsoup4:** Parsing HTML para extracción de precios y nombres de productos
- **pika:** Cliente Python para publicar/consumir mensajes en RabbitMQ
- **fastapi:** Framework REST asíncrono para el Servicio Agregador y la API
- **sqlalchemy:** ORM para interactuar con PostgreSQL de forma segura
- **pydantic:** Validación de esquemas de mensajes JSON y respuestas REST
- **apscheduler:** Scheduler para ejecutar scrapers periódicamente
- **uvicorn:** Servidor ASGI para ejecutar FastAPI en producción
- **psycopg2-binary:** Driver PostgreSQL nativo requerido por SQLAlchemy

3.7. Herramientas y Tipo de Software

Se optó por software libre en todos los componentes debido a su costo cero, amplia comunidad y madurez en producción.

- **Ventajas:** Sin licencias, libertad de modificación y portabilidad.
- **Desventajas:** Soporte sin SLA y responsabilidad propia en seguridad.

No hay costos de licencias; solo gastos de electricidad y mantención de hardware.

Tabla	Columna	Tipo	Descripción
supermercados	id	SERIAL PK	Identificador único autoincremental
	nombre	VARCHAR(50)	Nombre de la cadena supermercado
	url_base	TEXT	URL raíz del sitio a scrapear
	activo	BOOLEAN	Indica si el scraper está habilitado
productos	id	SERIAL PK	Identificador único autoincremental
	nombre	VARCHAR(200)	Nombre completo del producto
	categoría	VARCHAR(100)	Categoría (alimento, higiene, etc.)
	codigo_barra	VARCHAR(50)	Código EAN del producto
precios	id	SERIAL PK	Identificador único autoincremental
	producto_id	INT FK	Referencia a tabla productos
	supermercado_id	INT FK	Referencia a tabla supermercados
	precio	NUMERIC(10, 2)	Precio en pesos chilenos (CLP)
	registrado_en	TIMESTAMP	Fecha y hora exacta del registro

Figura 2. Modelo relacional y diccionario de datos

3.8. Frontend

Se incorpora un frontend en React 18, seleccionado por su amplio soporte comunitario y facilidad de integración con APIs REST. La interfaz permitirá buscar productos por nombre, comparar precios entre tiendas y consultar el historial de variaciones mediante gráficos. El diseño es responsive y compatible con dispositivos móviles.

4. Resultados Experimentales

Dado que el presente documento corresponde a la Iteración 1 del proyecto (fase de diseño), no existen resultados de implementación aún. Esta sección establece el diseño experimental que guiará las próximas iteraciones.

4.1. Diseño Experimental

A continuación se presentan las preguntas realizadas al cliente junto con los supuestos adoptados para el diseño y validación del sistema durante las Iteraciones 2 y 3:

1. **¿Con qué frecuencia deben actualizarse los precios?:** Cada 1 hora mediante APScheduler en cada scraper
2. **¿Cuántos productos distintos se monitorizarán?:** Hasta 500 productos por tienda (1.500 en total)
3. **¿Qué sistema operativo usan los servidores?:** Ubuntu 22.04 LTS en ambos servidores
4. **¿Cuántos servidores físicos hay disponibles?:** 2 servidores: 8 cores, 16 GB RAM, 500 GB SSD c/u
5. **¿Los sitios de supermercados permiten scraping?:** Scraping responsable con delays y User-Agent válido
6. **¿Se requiere historial de precios o solo el actual?:** Historial completo con marca temporal por registro
7. **¿Cuántos usuarios concurrentes usarán el frontend?:** Máximo 50 usuarios simultáneos en etapa piloto
8. **¿Se requiere autenticación de usuarios en el sistema?:** No se requiere autenticación en Iteración 1
9. **¿Se necesita notificación cuando un precio baja?:** No en Iteración 1; se evalúa para Iteración 3
10. **¿Los supermercados tienen API oficial disponible?:** No disponen de API; se trabaja con scraping web
11. **¿Qué categorías de productos son prioritarias?:** Alimentos, higiene personal y limpieza del hogar
12. **¿Se requiere soporte móvil además de escritorio?:** Diseño responsive compatible con dispositivos móviles

Las métricas a evaluar en iteraciones futuras incluirán: (1) latencia de propagación de un precio desde el scraper hasta la API REST, (2) tasa de éxito del scraping ante cambios en la estructura HTML de los sitios, (3) tiempo de respuesta de la API bajo carga concurrente de 50 usuarios, y (4) consistencia de los datos almacenados respecto a los precios reales.

5. Discusión

La elección del patrón Pub/Sub sobre RabbitMQ presenta ventajas claras frente a enfoques alternativos como la consulta directa (polling) desde el agregador a los scrapers. Al desacoplar productores y consumidor, el sistema tolera caídas temporales del agregador sin perder mensajes, y permite agregar nuevos scrapers (por ejemplo, para Santa Isabel o Tottus) sin modificar la arquitectura existente.

Una limitación relevante del diseño es la dependencia del scraping web: cualquier cambio en la estructura HTML de los sitios de supermercados puede interrumpir la recolección de datos. Para mitigar esto, los scrapers serán diseñados con selectores CSS flexibles y se implementará un sistema de alertas cuando la tasa de éxito de extracción caiga por debajo de un umbral.

Respecto a la escalabilidad, la arquitectura actual contempla dos servidores físicos. Si la demanda de usuarios aumenta considerablemente, Traefik permite incorporar instancias adicionales de la API REST sin cambios en la arquitectura de red. Del mismo modo, Docker Compose puede reemplazarse por Kubernetes en etapas futuras si se requiere orquestación más avanzada.

En cuanto a seguridad, la Iteración 3 incorporará TLS en la comunicación con RabbitMQ y JWT para autenticación de la API REST. Estos elementos no alteran la arquitectura fundamental, ya que fueron considerados desde el diseño inicial.

6. Conclusiones

Se ha presentado el diseño de un sistema distribuido para la comparación de precios en supermercados chilenos, basado en el patrón publicador/suscriptor, contenedores Docker y herramientas de software libre. La arquitectura propuesta resuelve los tres problemas identificados: dispersión de información, volatilidad de precios y ausencia de historial.

El diseño garantiza: (1) independencia entre scrapers, permitiendo fallos parciales sin afectar al sistema completo; (2) persistencia de mensajes ante caídas transitorias del agregador; (3) extensibilidad para incorporar nuevas cadenas de supermercado sin modificar la arquitectura existente.

Como trabajo futuro, las Iteraciones 2 y 3 abordarán la implementación completa del sistema, la incorporación de seguridad mediante TLS y JWT, y la validación experimental de las métricas de rendimiento definidas en el diseño experimental.

7. Acknowledgements

Referencias

1. Docker Inc.: Docker documentation. <https://docs.docker.com> (2024)
2. Hohpe, G., Woolf, B.: Enterprise Integration Patterns. Addison-Wesley (2003)
3. Kitchin, R.: The Data Revolution: Big Data, Open Data, Data Infrastructures and Their Consequences. SAGE Publications (2014)
4. Kleppmann, M.: Designing Data-Intensive Applications. O'Reilly Media (2017)
5. RabbitMQ Project: Rabbitmq documentation. <https://www.rabbitmq.com/documentation.html> (2024)
6. Ramírez, S.: Fastapi documentation. <https://fastapi.tiangolo.com> (2024)
7. The PostgreSQL Global Development Group: Postgresql 16 documentation. <https://www.postgresql.org/docs/16/> (2024)